SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

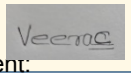
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1 – Apply (BL3)	**Assessment:**	Assessment Tool 2 – Dry Run Evaluation
Weightage:	35% of CIA	**Duration / Marks:**	60 mins / 100 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		
Student Name:	C Veera Bharadwaj Kumar	**Reg. No.:**	192565061
Section / Batch:	SLOT D/2025	**Date:**	08-10-2025

Code of Conduct

I C Veera Bharadwaj Kumar (192565061) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- Read each question carefully; answer all four questions (25 marks each, 100 in total).
- Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
- Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
- Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
- Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

Section / Topic	Focus	Question Nos.
1. Foundations and History of AI	Four approaches to AI, Turing Test, eras and milestones	**Q2**
2. Intelligent Agents: Agents and Environments	Agent, percepts, actions, agent function and program, PEAS	**Q1**
3. The Concept of Rationality	Performance measure, percept sequence, rational vs omniscient	**Q1**
4. The Nature of Environments	Task-environment properties and their classification	**Q1**
5. The Structure of Agents	Simple reflex and model-based reflex agent programs	**Q1**
6. Problem-Solving Agents	Goal formulation, problem formulation, search, execution	**Q2**
7. Problem Formulation	States, initial state, actions, goal test, path cost; belief states	**Q2, Q4**
8. Searching for Solutions	Search graph, frontier, explored set,	**Q3**

	node expansion	
9. Uninformed Search Strategies	Dry runs of BFS, DFS and uniform-	**Q3**

	cost search	
10. Search with Partial Information	Sensorless problems, belief states, percept update	**Q4**

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents | 15 minutes

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and −1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square.

Agent Y (model-based reflex): keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) Write the PEAS description (Performance measure, Environment, Actuators, Sensors) of this agent. **[4 Marks]**

(b) Classify the task environment on observability, number of agents, determinism, episodic/sequential, static/dynamic and discrete/continuous, with a one-phrase justification each. **[4 Marks]**

(c) Dry-run both agents for all 6 time steps. Complete the table (percept, action, and Y's internal state after the step). **[9 Marks]**

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1					
2					
3					
4					
5					
6					

(d) Compute the total performance of X and of Y over the 6 steps. **[3 Marks]**

(e) Interpret: which agent is rational? Both agents receive the identical percept [B, Clean] at step 4 – explain why they act differently, using the structure of the two agent programs, and distinguish a rational agent from an omniscient agent. **[5 Marks]**

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation | 15 minutes

Part A. Complete the table by classifying each system or idea under one of the four approaches to AI (thinking humanly, acting humanly, thinking rationally, acting rationally) and naming the era or milestone of AI history to which it belongs.

(a) Complete the table. **[8 Marks]**

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)		
General Problem Solver (Newell and Simon, 1957)		
Aristotle's syllogisms and formal logic		
An agent that does the right thing to maximise expected performance		

Part B. A problem-solving agent has two jugs: X of capacity 4 litres and Y of capacity 3 litres, both initially empty and without measuring marks. It may fill a jug from a tap, empty a jug on the ground, or pour one jug into the other until the source is empty or the target is full. The goal is to have exactly 2 litres in jug X.

(b) Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost. **[6 Marks]**

(c) Generate all successor states of (0, 0) and of (4, 0) (leave out actions that leave the state unchanged). **[3 Marks]**

(d) How many states does the state space contain? Write a shortest solution as a sequence of states and give its path cost. **[3 Marks]**

(e) Interpret: describe the four phases (goal formulation, problem formulation, search, execution) of a problem-solving agent for this task, and explain why the agent can execute the found solution without looking at new percepts. **[5 Marks]**

Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search | 15 minutes

Consider the weighted undirected graph below. The start node is S and the goal node is G. Use graph search with an explored set; expand neighbours alphabetically, test the goal when a node leaves the frontier, and break ties alphabetically. BFS and DFS ignore edge costs while searching, but path costs are measured with them.

Edge	Cost	Edge	Cost
S – A	1	B – D	1
S – B	4	C – G	6

A – C	2	D – G	2
A – D	5		

(a) Perform BFS from S (queue initially [S]). Do not add a node that is already explored or already in the queue. **[6 Marks]**

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1			
2			
3			
4			
5			
6			

(b) Perform DFS from S using a stack (initially [S]); push children so that the alphabetically first child is popped first. Write the stack with its top on the left. **[5 Marks]**

Iteration	Node Popped	Stack after Pushing Children
1		
2		
3		
4		

(c) Perform uniform-cost search from S. Write queue entries as Node:g(parent) and replace an entry whenever a cheaper path to a frontier node is found. **[7 Marks]**

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
1			
2			
3			
4			
5			
6			

(d) Complete the comparison table. **[4 Marks]**

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS				

DFS				
UCS				

(e) Interpret: explain why UCS removed as many nodes as BFS yet found a cheaper path, why DFS needed the fewest nodes here but cannot be trusted in general, and why an explored set is needed in graph search (use iteration 3 of BFS as an example). **[3 Marks]**

Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search) | 15 minutes

A vacuum agent lives in a two-square world (Left square, Right square), each square being Clean (C) or Dirty (D). Actions: Left, Right and Suck (Left in the left square and Right in the right square leave the agent where it is; Suck cleans the current square). Physical states are written (Agent, Left square, Right square):

State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)
S2	(R, D, D)	S6	(R, C, D)
S3	(L, D, C)	S7	(L, C, C)
S4	(R, D, C)	S8	(R, C, C)

(a) Initially the agent has no sensors and does not know its location or the dirt status. Formulate the problem over belief states: initial belief state, actions, goal test and path cost. **[3 Marks]**

(b) Apply the action sequence [Right, Suck, Left, Suck] and write the belief state after each action. **[6 Marks]**

Step	Action	Belief State after Action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	
2	Suck	
3	Left	
4	Suck	

(c) Now the agent has a local sensor that reports [location, status of the current square]. Starting again from the initial belief state, the first percept is [Left, Dirty]; the agent then performs Suck and perceives [Left, Clean]; it then performs Right and perceives [Right, Dirty]. Fill in the belief state after each action (prediction) and after each percept (update). **[8 Marks]**

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	
1	Suck		[Left, Clean]	
2	Right		[Right, Dirty]	

(d) How many different belief states are possible when there are 8 physical states? Why does this make the problem harder than searching over physical states? **[3 Marks]**

(e) Interpret: explain why the percepts in part (c) shrink the belief state, how this differs from the sensorless case in part (b), and why the sensorless plan can still be called rational. **[5 Marks]**

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Accuracy of Solution Path	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Q. No. / Criteria	Understanding of Search Problem	State Expansion and Dry Run Execution	Application of Search Algorithm	Accuracy of Solution Path	Accuracy of Solution Path	Sub. Total (20)	Total %
1							
2							
3							
4							
Total							

Faculty In-charge	Course Coordinator	Program Director

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

(a) PEAS description

Component	Description
Performance measure	+10 for each square cleaned (Suck on a Dirty square); −1 for each Left/Right move; evaluated over T = 6 time steps.
Environment	Two squares A and B (a 1×2 grid); each Clean or Dirty; initially both Dirty, dirt never reappears; agent starts in A.
Actuators	Wheels/motors (Left, Right), suction mechanism (Suck), and NoOp.
Sensors	Location sensor (A or B) and dirt sensor for the current square only → percept [location, status].

(b) Task-environment classification

Property	Classification	Justification
Observability	Partially observable	Only the dirt status of the current square is sensed; the other square's status is not visible.
Number of agents	Single agent	Only the vacuum agent acts; nothing else affects the world.
Determinism	Deterministic	Next state is fully determined by current state and action (dirt never reappears).
Episodic / Sequential	Sequential	Current action (move/clean) affects later percepts and future score.
Static / Dynamic	Static	World does not change while the agent deliberates; no new dirt.

Discrete / Continuous	Discrete	Finite squares, percepts, actions and discrete time steps.

(c) Dry run of Agents X and Y for T = 6

Initial: agent in A; A = Dirty, B = Dirty. Y's known-clean set starts as { }. Y adds a square to the set when it sucks it or perceives it Clean.

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares (after step)
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right (B not known clean)	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp (both known clean)	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

Note: X's location changes (A→B→A→B→A), so from step 5 X's percepts are [A, Clean] and [B, Clean]; Y stays in B.

(d) Total performance

Agent	Squares cleaned (+10 each)	Moves (−1 each)	Total
X	2 Suck → +20	Steps 2,4,5,6 = 4 moves →	**+16**

		−4	
Y	2 Suck → +20	Step 2 only = 1 move → −1	**+19**

Final answer: Performance(X) = 16, Performance(Y) = 19.

(e) Interpretation

- **Which is rational?**

  Agent Y is rational: for every percept sequence it chooses the action that maximises expected performance given its built-in knowledge and memory (it stops moving once both squares are known clean, avoiding −1 penalties). X is not rational over the whole task – it keeps shuttling after the job is done and loses 3 extra points (19 vs 16).

- **Why different actions on the identical percept [B, Clean] at step 4?**

  X is a simple reflex agent: its program is a condition–action rule using only the current percept ("if Clean, move to the other square"), with no memory, so it always moves Left. Y is a model-based reflex agent: it maintains an internal state (known-clean = {A, B}) updated from the percept history and uses it in its rule, so the same percept yields NoOp.

- **Rational vs omniscient:** An omniscient agent knows the actual outcome of its actions in advance and would always act perfectly (e.g., know B was Dirty at step 2). A rational agent maximises expected performance using the percepts and knowledge it has; Y did not know B was dirty at step 2 but moving Right was still rational. Rationality ≠ perfection; omniscience is impossible in practice.

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

(a): Classification table

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed	Acting humanly	Gestation of AI (1943–1955): Turing's

(1950)		"Computing Machinery and Intelligence"
General Problem Solver (Newell & Simon, 1957)	Thinking humanly (cognitive modelling – compares reasoning steps to human problem-solving)	Early enthusiasm, great expectations (1952–1969)
Aristotle's syllogisms and formal logic	Thinking rationally ("laws of thought")	Ancient philosophical/logical foundations of AI (c. 350 BC); later the logicist tradition
An agent that does the right thing to maximise expected performance	Acting rationally (rational-agent approach)	Emergence of intelligent agents (1995–present)

(b): Problem formulation – water-jug problem

Component	**Formulation**
State representation	(x, y): x = litres in jug X ($0 \le x \le 4$), y = litres in jug Y ($0 \le y \le 3$).
Initial state	(0, 0)
Actions	Fill X; Fill Y; Empty X; Empty Y; Pour X→Y; Pour Y→X.
Transition model	Fill X: (x,y)→(4,y). Fill Y: (x,y)→(x,3). Empty X: →(0,y). Empty Y: →(x,0). Pour X→Y: t = min(x, 3−y) → (x−t, y+t). Pour Y→X: t = min(y, 4−x) → (x+t, y−t).
Goal test	x = 2, i.e. state (2, y) for any y.
Path cost	1 per action (path cost = number of actions).

(c) Successor states

State	Successors (unchanged-state actions omitted)
(0, 0)	Fill X → **(4, 0)**; Fill Y → **(0, 3)**. (Empty X, Empty Y, Pour X→Y, Pour Y→X leave state unchanged.)
(4, 0)	Fill Y → **(4, 3)**; Empty X → **(0, 0)**; Pour X→Y (t = min(4,3) = 3) → **(1, 3)**. (Fill X, Empty Y, Pour Y→X unchanged.)

(d) State-space size and shortest solution

- Possible states = 5 × 4 = **20** (x = 0..4, y = 0..3). Of these, **14 are reachable** from (0, 0): (0,0),(0,1),(0,2),(0,3),(1,0),(1,3),(2,0),(2,3),(3,0),(3,3),(4,0),(4,1),(4,2),(4,3). The unreachable ones are (1,1),(1,2),(2,1),(2,2),(3,1),(3,2).
- Shortest solution (BFS shows no solution with fewer than 6 actions):

(0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0)

Actions: Fill Y, Pour Y→X, Fill Y, Pour Y→X, Empty X, Pour Y→X. **Path cost = 6.** (Alternative of equal cost: (0,0)→(4,0)→(1,3)→(1,0)→(0,1)→(4,1)→(2,3).)

(e) Interpretation: four phases of the problem-solving agent

- **Goal formulation:** limit the objective – reach any state with exactly 2 litres in jug X (goal set {(2,0),(2,3)}).
- **Problem formulation:** decide states (x, y), initial state (0,0), the six actions, transition model, goal test and unit step cost (as in part b).
- **Search:** examine sequences of actions (e.g., BFS) to find the action sequence leading from (0,0) to a goal state – the 6-step solution above.
- **Execution:** carry out the actions of the solution one by one: Fill Y, Pour Y→X, Fill Y, Pour Y→X, Empty X, Pour Y→X.

- **Why no new percepts are needed:** the environment is fully observable, deterministic, static, known and discrete, so the agent can predict every outcome exactly. It can execute the plan "eyes closed" (open-loop), ignoring percepts, because the result of each action is certain.

Question 3: Searching for Solutions – Uninformed Search Strategies

Graph edges: S–A 1, S–B 4, A–C 2, A–D 5, B–D 1, C–G 6, D–G 2. Adjacency (alphabetical): S: A,B | A: C,D,S | B: D,S | C: A,G | D: A,B,G | G: C,D.

(a) BFS (FIFO queue)

Iter	Node Dequeued	Queue after Expansion	Explored Set

0 (init)	–	[S]	{ }
1	S	[A, B]	{S}
2	A	[B, C, D] (S explored)	{S, A}
3	B	[C, D] (S explored; D already in queue → not added)	{S, A, B}
4	C	[D, G] (A explored)	{S, A, B, C}
5	D	[G] (A, B explored; G already in queue → not added)	{S, A, B, C, D}
6	G	Goal test passes → STOP	{S, A, B, C, D}

BFS: G was inserted by C (expanded before D), so path = S → A → C → G, cost 1 + 2 + 6 = 9. Nodes removed: 6 (S, A, B, C, D, G).

(b) DFS (stack, top on the left)

Iter	Node Popped	Stack after Pushing Children	Remark
0 (init)	–	[S]	
1	S	[A, B]	push B then A; A on top

2	A	[C, D, B]	S explored; push D then C
3	C	[G, D, B]	A explored; push G
4	G	Goal test passes → STOP	goal found

DFS: path = S → A → C → G, cost 9. Nodes removed: 4 (S, A, C, G).

(c) Uniform-cost search (priority queue ordered by g, Node:g(parent))

Iter	Node Expanded	g(n)	Priority Queue after Expansion
0 (init)	–	–	[S:0]
1	S	0	[A:1(S), B:4(S)]
2	A	1	[C:3(A), B:4(S), D:6(A)]
3	C	3	[B:4(S), D:6(A), G:9(C)]
4	B	4	[D:5(B), G:9(C)] (D improved 6→5 via B)
5	D	5	[G:7(D)] (G improved 9→7 via D)
6	G	7	Goal test passes → STOP

UCS: path = S → B → D → G, cost 4 + 1 + 2 = 7. Nodes removed: 6 (S, A, C, B, D, G).

(d) Comparison table

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS	S → A → C → G	9	6	No

DFS	S → A → C → G	9	4	No
UCS	S → B → D → G	7	6	Yes

(e) Interpretation

- **UCS vs BFS:** both removed 6 nodes, but BFS orders nodes by depth (number of edges) and ignores costs, so it stopped at the first 3-edge path to G (cost 9). UCS orders by path cost g(n), so it updated D and G to cheaper values via B and found the optimal path (cost 7).
- **DFS:** it needed the fewest nodes (4) only because the first branch it chose happened to lead to G; it is not optimal (cost 9 > 7), is incomplete in infinite/looping spaces, and can go very deep, so it cannot be trusted in general.
- **Explored set:** in BFS iteration 3, node B's neighbours are S (already explored) and D (already in the queue). Without the explored set/frontier check, S would be re-added (cycle in an undirected graph → infinite loop) and D duplicated, wasting time and memory.

Question 4: Search with Partial Information – Belief States

States: S1 (L,D,D), S2 (R,D,D), S3 (L,D,C), S4 (R,D,C), S5 (L,C,D), S6 (R,C,D), S7 (L,C,C), S8 (R,C,C). Goal states are S7 and S8 (both squares clean).

(a) Belief-state formulation

Component	Formulation
Initial belief state	{S1, S2, S3, S4, S5, S6, S7, S8} – agent has no information.
Actions	Left, Right, Suck. Result of an action on a belief state b = set of results of the action applied to every state in b.

Goal test	Every state in the belief state is a goal state, i.e. belief ⊆ {S7, S8}.
Path cost	1 per action.

(b) Sensor less: [Right, Suck, Left, Suck]

Step	Action	Belief State after Action	Working
0 (start)	–	{S1,S2,S3,S4,S5,S6,S7,S8}	
1	Right	{S2, S4, S6, S8}	S1,S2→S2; S3,S4→S4; S5,S6→S6; S7,S8→S8
2	Suck	{S4, S8}	S2→S4; S4→S4; S6→S8; S8→S8
3	Left	{S3, S7}	S4→S3; S8→S7
4	Suck	{S7}	S3→S7; S7→S7 → goal (⊆ {S7,S8})

Final answer: after 4 actions the belief state is {S7} – the world is certainly in (L, C, C) without any sensing.

(c) With local sensor: prediction and update

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0 (start)	–	{S1,S2,S3,S4,S5,S6,S7,S8}	[Left, Dirty]	{S1, S3}
1	Suck	{S5, S7}	[Left, Clean]	{S5, S7}
2	Right	{S6, S8}	[Right, Dirty]	{S6}

Working: Percept [Left, Dirty] keeps states with agent in L and left square D: S1, S3. Suck: S1→S5, S3→S7. Percept [Left, Clean]: both S5 and S7 have agent L and left clean, so no change. Right: S5→S6, S7→S8. Percept [Right, Dirty]: only S6 has right square dirty (S8 is clean) → {S6}. A final Suck gives S8 (goal).

(d) Number of belief states

- A belief state is any subset of the 8 physical states: 2^8 = **256** (255 non-empty).
- In general 2^N for N physical states – exponentially larger than the N physical states. The search must reason about sets of states, so the space grows exponentially, and a belief state may itself contain many states, making storage, expansion and goal testing harder.

(e) Interpretation

- **Why percepts shrink the belief state:** each percept is consistent with only some physical states. The update step discards states that could not have produced the percept ([Left, Dirty] removed 6 of 8 states; [Right, Dirty] left only S6). Percepts supply information and reduce uncertainty.
- **Difference from the sensorless case:** with no sensors there is only the prediction step; the belief state shrinks only because actions coerce the world (Right, Suck, Left, Suck) into fewer states, and the plan must work for every state in the belief set. With a sensor the agent alternates prediction and update, can branch on percepts (contingent plan), and reach certainty (a singleton) faster.
- **Why the sensorless plan is rational:** with no percepts the agent cannot do better; [Right, Suck, Left, Suck] guarantees the goal {S7} from every possible start, at minimal cost (4 actions), so it maximises expected performance given the agent's knowledge. Rationality depends on available information, not on omniscience.